MAŁOPOLSKA UCZELNIA PAŃSTWOWA IMIENIA ROTMISTRZA WITOLDA PILECKIEGO W OŚWIĘCIMIU

Instytut: Nauk Inżynieryjno-Technicznych

Kierunek studiów: Informatyka

Poziom studiów: Studia pierwszego stopnia

Moduł specjalnościowy: Informatyka w przedsiębiorstwie

Forma studiów: stacjonarna

Krystian Zając

nr albumu 8795

Aplikacja Mobilna GPS i Krokomierz

Projekt na przedmiot

systemy mobilne

mgr. inż. Artur Pelo

Oświęcim 2026

Abstrakt

Imię i nazwisko autora pracy	Krystian Zając
Rok urodzenia autora pracy	2003
Stopień/tytuł zawodowy, imię i nazwisko promotora pracy	mgr inż. Artur Pelo
Instytut/Kierunek/Poziom/ Moduł specjalnościowy	Instytut Nauk Inżynieryjno-Technicznych / Informatyka / Inżynierskie / Informatyka w przedsiębiorstwie
Data realizacji projektu	od 18.03.2026 do 1.04.2026
Termin zaliczenia projektu	1.04.2026
Nazwa przedmiotu	systemy mobilne
Rok studiów/semestr	rok III, semestr VI

Spis treści

Wstęp 4
1. Cel i zakres pracy 5
1.1. Cel pracy 5
1.2. Cele szczegółowe 5
1.3. Zakres pracy 5
2. Analiza wymagań i projekt rozwiązania 6
2.1. Wymagania funkcjonalne i niefunkcjonalne 6
2.2. Architektura rozwiązania 6
2.3. Struktura projektu 6
3. Opis działania aplikacji 7
3.1. Środowisko i narzędzia 7
3.2. Struktura projektu 7
3.3. Diagram UML 7
3.4. UI ekranu głównego 8
3.5. Implementacja krokomierza 9
3.6. Wznowienie/wstrzymanie krokomierza 9
3.7. Wyświetlanie aktualnych danych w czasie rzeczywistym 10
3.8. Implementacja mapy 11
3.9. Zapis danych 12
3.10. UI dziennika 13
3.11. Implementacja blokowa dziennika 14
4. Instrukcja instalacji i użytkowania 15
4.1. Wymagania sprzętowe 15
4.2. Instrukcja instalacji 15
4.3. Instrukcja użytkowania 15
5. Testy i weryfikacja 16
5.1. Testy funkcjonalne 16
5.2. Testy niefunkcjonalne 16
5.3. Wyniki testów 16
Netografia 17
Spis rysunków 18
Spis tabel 19

Wstęp

Tematem projektu jest opracowanie prostej alpikacji mobilnej umożliwiającej szacowanie aktualnej lokalizacji urządzenia oraz pomiar liczby kroków pomiędzy sekcjami Start i Stop. Zgodnie z założeniami system powinien wykonywać swoją funkcję korzystając z sensora GPS, akcelerometra, zegara oraz lokalnej bazy danych umożliwiającej zapis zebranych informacji.

1. Cel i zakres pracy

1.1. Cel pracy

Celem pracy jest utworzenie aplikacji mobilnej w środowisku (*MIT App Inventor* 2026) umożliwiającej śledzenie lokalizacji GPS urządzenia oraz pomiar liczby kroków pomiędzy sekcjami Start i Stop. System powinien zapisywać dane do lokalnej bazy danych TinyDB i umożliwiać przeglądanie dziennika zapisów.

1.2. Cele szczegółowe

W ramach realizacji celu głównego wyznaczono następujące cele szczegółowe:

- nauka obsługi środowiska MIT App Inventor,
- opracowanie czytelnego i łatwego w obsłudze interfejsu użytkownika,
- implementacja krokomierza korzystając z akcelerometra,
- pozyskiwanie danych z modułu GPS oraz weryfikacja ich dokładności,
- wyświetlanie danych o dacie, godzinie, lokalizacji, jakości sygnału oraz liczbie kroków,
- implementacja bazy danych oraz wyświetlanie dziennika,
- testowanie oraz konfiguracja krokomierza w celu uzyskania dokładnych pomiarów

1.3. Zakres pracy

Zakres pracy obejmuje:

- zaprojektowanie UI aplikacji,
- opracowanie logiki systemu przy pomocy bloków w MIT App Inventor,
- testowanie działania aplikacji w prawdziwym środowisku,
- przygotowanie dokumentacji zgodnej z wymaganiami projektowymi.

2. Analiza wymagań i projekt rozwiązania

Rozdział przedstawia zestaw wymagań, opis architektury oraz struktury projektu.

2.1. Wymagania funkcjonalne i niefunkcjonalne

Tabela 1
Wymagania funkcjonalne

ID	Opis	Priorytet
FR-001	Pozyskiwanie danych z sensora GPS z dokładnością ±5m	Wysoki
FR-002	Pomiar kroków za pomocą akcelerometru	Wysoki
FR-003	Zapis danych do lokalnej bazy SQLite	Wysoki
FR-004	Wyświetlanie dzinnika zapisów z datą, godziną, lokalizacją i liczbą kroków	Wysoki

*Źródło: pelo.com.pl (*System zasobów - eStudent *2026)*

Tabela 2
Wymagania niefunkcjonalne

ID	Opis	Priorytet
NFR-001	Czas odpowiedzi UI mniejszy niż 200ms	Wysoki
NFR-002	Zużycie baterii mniejsze niż 10% na godzinę	Wysoki
NFR-003	Kompatybilność z Android 8.0+	Wysoki

*Źródło: pelo.com.pl (*System zasobów - eStudent *2026)*

2.2. Architektura rozwiązania

Aplikacja została zaprojektowana w oparciu o architekturę zdarzeniową, charakterystyczną dla środowisk programowania wizualnego. Logika aplikacji realizowana jest za pomocą połączonych bloków reagujących na zdarzenia generowane przez komponenty systemowe.

2.3. Struktura projektu

Projekt składa się z pliku głównego dokumentu oraz rozdziałów tematycznych, uzupełnionych o pliki bibliografii i zasoby graficzne. Struktura jest zgodna z przyjętymi standardami prac inżynierskich i ułatwia dalszą rozbudowę.

3. Opis działania aplikacji

Rozdział opisuje logikę funkcjonowania systemu poprzez opisanie jego poszczególnych funkcjonalności oraz przedstawienie stosów bloków odpowiedzialnych za ich działanie.

3.1. Środowisko i narzędzia

Implementacja opiera się na środowisku programistycznym MIT App Inventor służącym do tworzenia aplikacji mobilnych w oparciu o programowanie wizualne. Zamiast kodu wykorzystuje się bloki, które łączy się w logiczne struktury odpowiadające działaniu aplikacji.

3.2. Struktura projektu

Projekt zawiera plik instalacyjny w formacie APK oraz plik ReadMe w formacie md. Umożliwia to użytkownikowi końcowemu instalację aplikacji oraz dostęp do podstawowych instrukcji i informacji.

3.3. Diagram UML

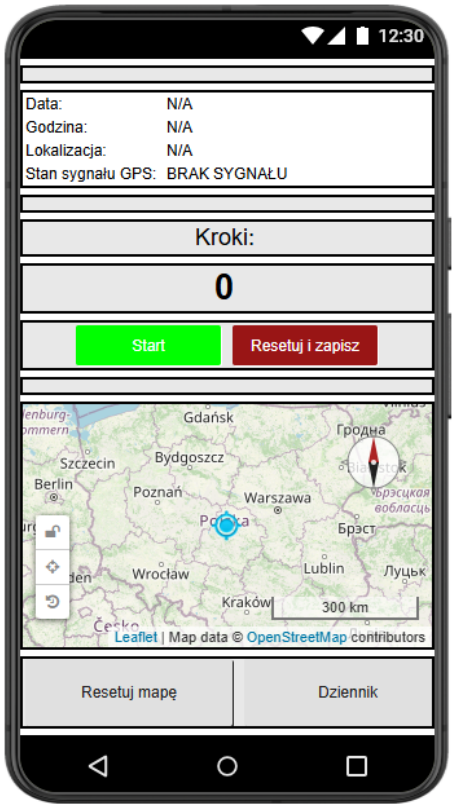
Rysunek 1. Diagram przypadków użycia (import z pliku PNG)

*Źródło: opracowanie własne (*PlantUML *2026)*

3.4. UI ekranu głównego

Jak pokazano na rysunku 2, poniżej znajduje się podgląd rozmieszczenia elementów interfejsu na ekranie głównym w środowisku MIT AI. W skład interfejsu tego ekranu wchodzą:

- tabela wyświetlająca dane o aktualnej dacie, godzinie, lokalizacji oraz stanie sygnału GPS,
- licznik kroków,
- przycisk start/stop,
- przycisk resetujący licznik i zapisujący dane.
- mapa
- przycisk resetujący wyśrodkowanie oraz przybliżenie mapy,
- przycisk umożliwiający przejście do dziennika z zapisami.



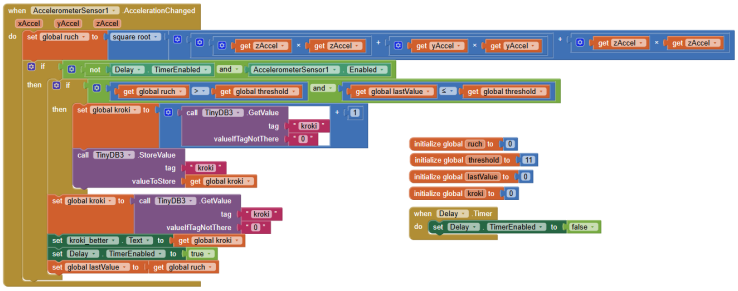
Rysunek 2. Rozmieszczenie elementów na ekranie głównym w środowisku MIT AI (import z pliku PNG)

Źródło: opracowanie własne (MIT App Inventor *2026)*

Na rysunku 2 zaprezentowano wizualizację ekranu głównego aplikacji z poziomu środowiska MIT AI. Wizualizacja nie reprezentuje faktycznego wyglądu uruchomionej aplikacji.

3.5. Implementacja krokomierza

Środowisko (*MIT App Inventor* 2026) oferuje gotowy krokomierz, natomiast w celu pogłębienia znajomości tego środowiska, opracowano krokomierz od podstaw przy pomocy akcelerometra. Takie rozwiązanie umożliwia większą kontrolę nad dokładnością wykrywania kroków oraz ułatwia dalszą rozbudowę i edycję krokomierza. Na rysunku 3, poniżej znajduje się zestaw bloków wizualizujący logikę działania krokomierza.



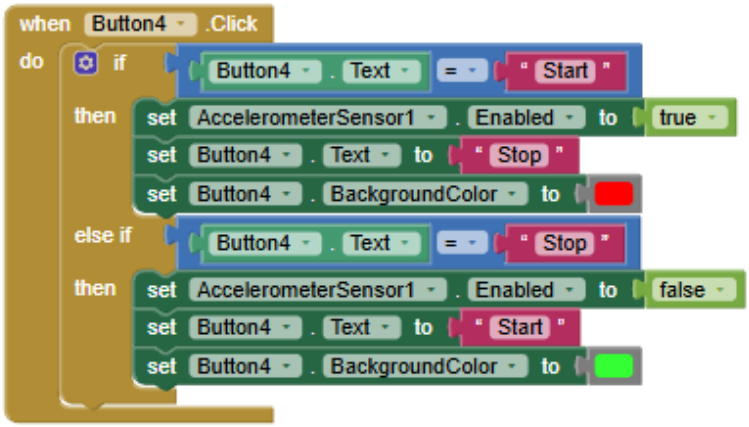
Rysunek 3. Implementacja blokowa krokomierza (import z pliku PNG)

*Źródło: opracowanie własne (*MIT App Inventor *2026)*

Na rysunku 3 zaprezentowano implementację blokową krokomierza.

3.6. Wznowienie/wstrzymanie krokomierza

Jak pokazano na rysunku 4, poniżej znajduje się stos bloków odpowiedzialny za działanie przycisku umożliwiającego rozpoczęcie zliczania kroków oraz jego zatrzymanie.



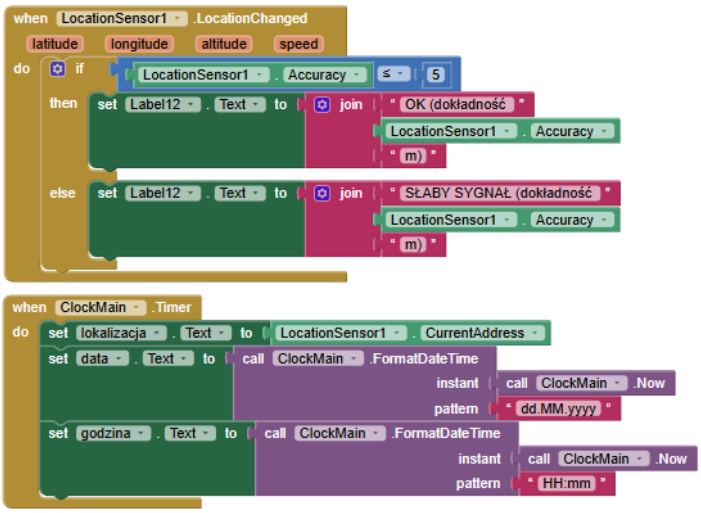
Rysunek 4. Implementacja blokowa przycisku Start/Stop (import z pliku PNG)

*Źródło: opracowanie własne (*MIT App Inventor *2026)*

Na rysunku 4 zaprezentowano implementację blokową przycisku Start/Stop. W zależności od tego jaki tekst znajduje się na przycisku zmienia on swoją funkcję oraz wygląd.

3.7. Wyświetlanie aktualnych danych w czasie rzeczywistym

Jak pokazano na rysunku 5, poniżej znajduje się stos bloków odpowiedzialny za wyświetlanie informacji o aktualnej dacie, godzinie, lokalizacji oraz stanie sygnału GPS w czasie rzeczywistym.



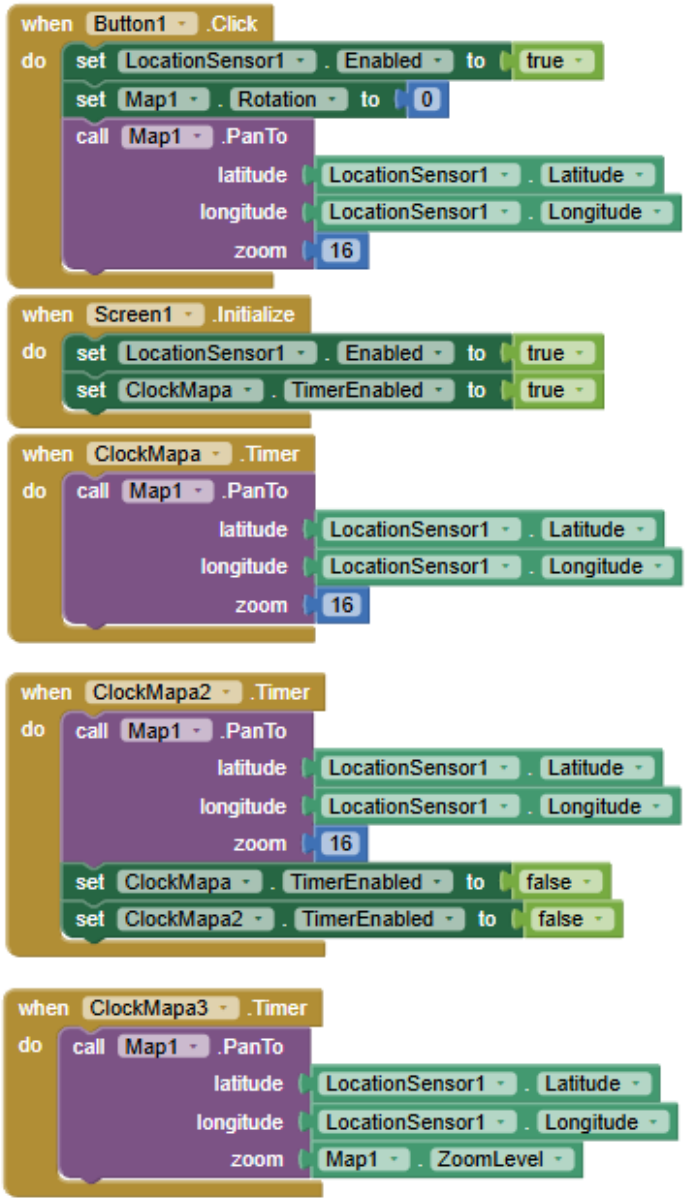
Rysunek 5. Implementacja blokowa wyświetlania danych (import z pliku PNG)

Źródło: opracowanie własne (MIT App Inventor *2026)*

Na rysunku 5 zaprezentowano implementację blokową wyświetlania danych.

3.8. Implementacja mapy

Jak pokazano na rysunku 6, poniżej znajduje się stos bloków odpowiedzialny za działanie mapy.



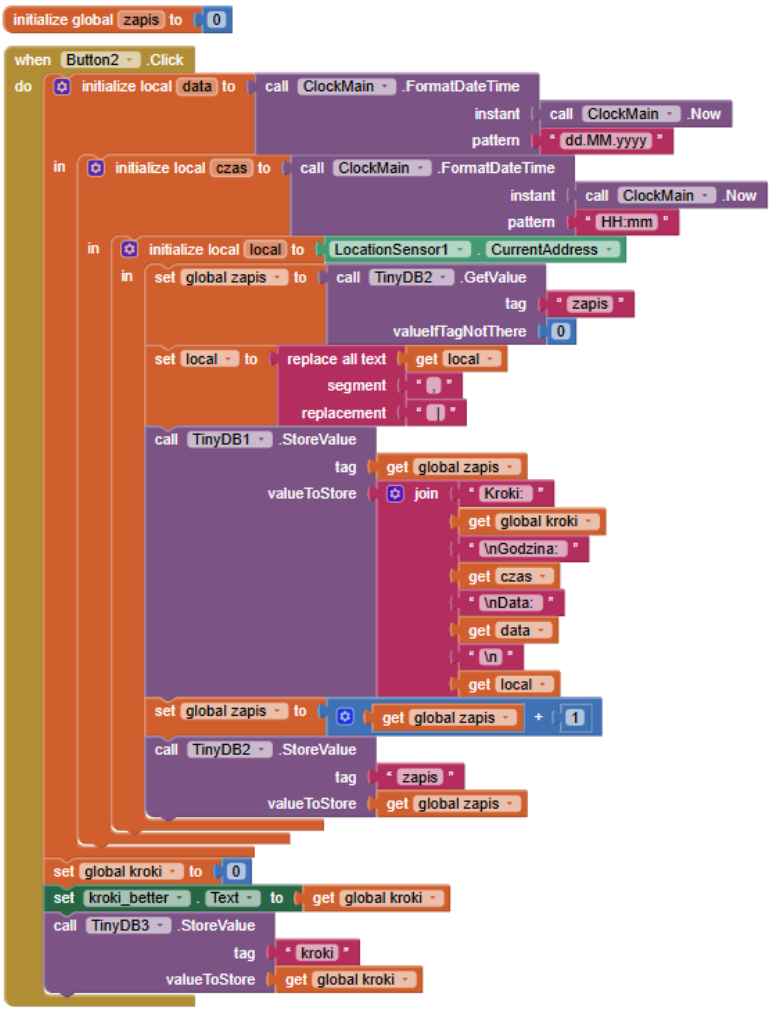
Rysunek 6. Implementacja blokowa mapy (import z pliku PNG)

Źródło: opracowanie własne (MIT App Inventor *2026)*

Na rysunku 6 zaprezentowano implementację blokową mapy zintegrowanej z czujnikiem GPS. Mapa automatycznie się centruje do miejsca aktualnej lokalizacji.

3.9. Zapis danych

Jak pokazano na rysunku 7, poniżej znajduje się stos bloków odpowiedzialny za zapis informacji do lokalnej bazy danych TinyDB.



Rysunek 7. Implementacja blokowa zapisu danych (import z pliku PNG)

Źródło: opracowanie własne (MIT App Inventor *2026)*

Na rysunku 7 zaprezentowano strukturę systemu w ujęciu ogólnym.

3.10. UI dziennika

Jak pokazano na rysunku 8, poniżej znajduje się podgląd rozmieszczenia elementów interfejsu na ekranie dziennika w środowisku MIT AI. W skład interfejsu tego ekranu wchodzą:

- pasek tytułowy,
- przestrzeń w której będzie się wyświetlać lista wpisów,
- przycisk powrotu do ekranu głównego,
- przycisk usuwający wpisy.

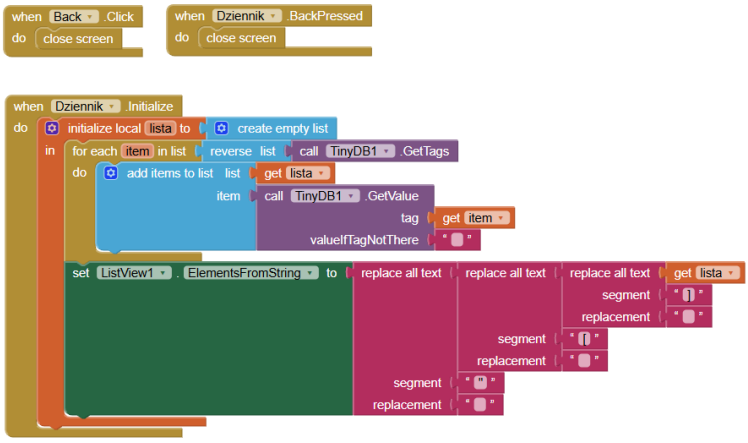
Rysunek 8. Implementacja blokowa wyświetlania dziennika (import z pliku PNG)

*Źródło: opracowanie własne (*MIT App Inventor *2026)*

Na rysunku 8 zaprezentowano wizualizację ekranu dziennika z poziomu środowiska MIT AI. Wizualizacja nie reprezentuje faktycznego wyglądu uruchomionej aplikacji.

3.11. Implementacja blokowa dziennika

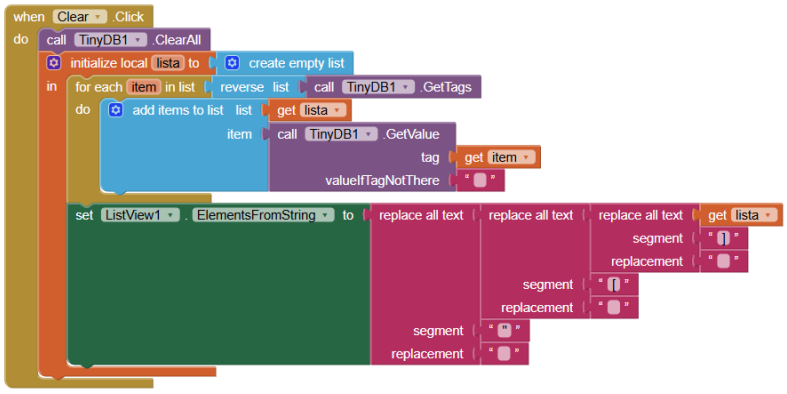
Na rysunku 9 zaprezentowano implementację blokową wyświetlania dziennika.



Rysunek 9. Implementacja blokowa wyświetlania dziennika (import z pliku PNG)

*Źródło: opracowanie własne (*MIT App Inventor *2026)*

Na rysunku 10 zaprezentowano implementację blokową czyszczenia dziennika.



Rysunek 10. Implementacja blokowa czyszczenia dziennika (import z pliku PNG)

*Źródło: opracowanie własne (*MIT App Inventor *2026)*

4. Instrukcja instalacji i użytkowania

Rozdział zawiera informacje potrzebne do instalacji i używkowania aplikacji.

4.1. Wymagania sprzętowe

Telefon, tablet lub emulator:

- System operacyjny Android w wersji 2.1 lub nowszej,
- Działający sensor GPS i akcelerometr,
- Przynajmniej 30MB przestrzeni dyskowej,
- Poprawnie skonfigurowana data i godzina w systemie.

4.2. Instrukcja instalacji

- Zezwól urządzeniu na pobieranie i instalację aplikacji z nieznanych źródeł,
- Pobierz plik APK,
- Uruchom plik i zainstaluj aplikację,
- Uruchom aplikację,
- Zezwól aplikacji na dostęp do lokalizacji,

4.3. Instrukcja użytkowania

W celu rozpoczęcia pomiaru kroków uruchom aplikację, zezwól na dostęp do lokalizacji urządzenia oraz kliknij zielony przycisk "Start". Aby wstrzymać licznik kroków kliknij czerwony przycisk "Stop", który znajduje się w miejscu poprzednio klikniętego przycisku "Start". Aby zapisać dane kliknij czerwony przycisk "Resetuj i zapisz". Następnie możesz przejść do dziennika, aby zobaczyć nowy wpis.

5. Testy i weryfikacja

Rozdział przedstawia sposób sprawdzenia poprawności działania aplikacji oraz ocenę uzyskanych rezultatów. Testy skupiają się na weryfikacji poprawności działania oraz zgodności z wymaganiami funkcjonalnymi i niefunkcjonalnymi.

5.1. Testy funkcjonalne

W ramach testów funkcjonalnych sprawdzono poprawność realizacji kluczowych funkcji systemu, takich jak:

- pozyskiwanie danych lokalizacyjnych z wykorzystaniem modułu GPS wraz z weryfikacją ich dokładności,
- pomiar liczby kroków z użyciem akcelerometru,
- sterowanie procesem pomiaru (rozpoczęcie i zakończenie sesji),
- zapis danych do lokalnej bazy danych,
- prezentacja historii zapisów w formie dziennika.

5.2. Testy niefunkcjonalne

Testy niefunkcjonalne obejmowały ocenę wydajności, responsywności oraz kompatybilności aplikacji. W szczególności zweryfikowano:

- czas reakcji interfejsu użytkownika, który pozostaje na akceptowalnym poziomie,
- zużycie zasobów urządzenia, w tym baterii, podczas ciągłego działania aplikacji,
- poprawność działania na urządzeniach z systemem Android w wymaganej wersji.

5.3. Wyniki testów

Przeprowadzone testy potwierdziły stabilność działania aplikacji oraz zgodność jej funkcjonalności z przyjętymi wymaganiami. System poprawnie realizuje proces pozyskiwania i przetwarzania danych lokalizacyjnych oraz pomiaru aktywności użytkownika, zapewniając jednocześnie odpowiednią jakość działania i użyteczność.

Netografia

MIT App Inventor (2026). Ang. URL: https://appinventor.mit.edu/ (dostęp 31.03.2026).
PlantUML (2026). Ang. URL: https://www.plantuml.com/ (dostęp 31.03.2026).
System zasobów - eStudent (2026). URL: https://pelo.com.pl/ (dostęp 31.03.2026).

Spis rysunków

Rysunek 1. Diagram przypadków użycia (import z pliku PNG) 7
Rysunek 2. Rozmieszczenie elementów na ekranie głównym w środowisku MIT AI (import z pliku PNG) 8
Rysunek 3. Implementacja blokowa krokomierza (import z pliku PNG) 9
Rysunek 4. Implementacja blokowa przycisku Start/Stop (import z pliku PNG) . . 9
Rysunek 5. Implementacja blokowa wyświetlania danych (import z pliku PNG) . . 10
Rysunek 6. Implementacja blokowa mapy (import z pliku PNG) 11
Rysunek 7. Implementacja blokowa zapisu danych (import z pliku PNG) 12
Rysunek 8. Implementacja blokowa wyświetlania dziennika (import z pliku PNG) 13
Rysunek 9. Implementacja blokowa wyświetlania dziennika (import z pliku PNG) 14
Rysunek 10. Implementacja blokowa czyszczenia dziennika (import z pliku PNG) . 14

Spis tabel

Tabela 1. Wymagania funkcjonalne 6
Tabela 2. Wymagania niefunkcjonalne 6